```
var_dump(dechex($int));
```

```
Output:
string(4) "ffff"
```

We can see that the decimal value *65535* is equivalent to the hexadecimal number *ffff*, which would be stored in memory as the bytes `0xff 0xff`. The specific bytes stored for a single PHP integer would be at least `0x00 0x00 0xff 0xff` (for a 32-bit system) – see Chapter 7, "Numbers," for more information.

We can do something similar to get the byte values for the string `'65535'`, but first we need to use the built-in function `ord()`, which converts a character to its ordinal value in the ASCII codepage as a decimal. (Given that we've already seen how ASCII encodes characters to single bytes, it should already be apparent that the byte values for the numerical string are not going to be equivalent to the byte values of the integer, based on size alone!)

```
$byte1 = dechex(ord('6'));
$byte2 = dechex(ord('5'));
$byte3 = dechex(ord('5'));
$byte4 = dechex(ord('3'));
$byte5 = dechex(ord('5'));

var_dump($byte1 . $byte2 . $byte3 . $byte4 . $byte5);
```

```
Output:
string(10) "3635353335"
```

The code above might look a little intimidating to a novice programmer, but fear not, it's relatively straightforward. We're making five calls to `ord()`, one for each character in our string, because the `ord()` function only works with single characters. Each of those `ord()` calls is wrapped in a call to `dechex()` because `ord()` gives the decimal codepage value, and it's the hexadecimal value we need for comparison. As you can see, we're dealing with two very different pieces of data: *65535* is stored as `0x00 0x00 0xff 0xff` (on a 32-bit system), and `'65535'` is stored as `0x36 0x35 0x35 0x33 0x35`.

When we used an integer in a string operation (and a string in a mathematical operation), PHP transformed the data. This was necessary because `0xff 0xff` expressed as a string is `'ÿÿ'`, and `0x36 0x35 0x35 0x33 0x35` expressed as an integer is *232820912949* (or something of similar magnitude depending on CPU architecture).